UNIVERSITÉ HASSAN 1ER

ÉCOLE NATIONALE DE COMMERCE ET DE GESTION DE SETTAT

COMPTE RENDU DE PROJET — Projet N° 14

Prédiction du Turnover des Auditeurs Internes par Machine Learning

Module Intelligence Artificielle — Filière CAC / Option Contrôle, Audit et Conseil — L3 S8

Intitulé :	Prédiction du Turnover des Auditeurs Internes par ML
Module :	Intelligence Artificielle
Filière :	CAC — Contrôle, Audit et Conseil — L3 S8
Groupe :	CAC 2
Réalisé par :	KIHEL Hajar / HOUMADI Nada
Encadrant :	Dr. A. Larhlimi
Période :	Année Universitaire 2025–2026
Date de remise :	Mars 2026

Équipe Projet

Prédiction du Turnover des Auditeurs Internes par Machine Learning

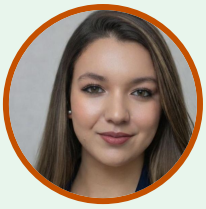
KIHEL Hajar

N° Apogée : 24010389

Filière : Contrôle, Audit & Conseil

Groupe : CAC 2

Université : ENCG Settat



HOUMADI Nada

N° Apogée : 24010393

Filière : Contrôle, Audit & Conseil

Groupe : CAC 2

Université : ENCG Settat

Encadrant : Dr. A. Larhlimi **Module :** Intelligence Artificielle — ENCG Settat **Année Universitaire :** 2025 – 2026

SOMMAIRE

I. INTRODUCTION

I.1 Contexte et problématique

I.2 Objectifs du projet

I.3 Méthodologie adoptée

II. DÉVELOPPEMENT

II.1 Présentation du dataset IBM HR Analytics

II.2 Exploration et Analyse des Données (EDA)

II.3 Prétraitement des Données

II.4 Modèles de Machine Learning testés

II.5 Interprétabilité avec SHAP

III. RÉSULTATS ET DISCUSSION

III.1 Tableau comparatif des performances

III.2 Analyse SHAP et importance des variables

III.3 Simulation individuelle

III.4 Analyse critique et interprétation

III.5 Limites et axes d'amélioration

IV. RECOMMANDATIONS MÉTIER

IV.1 Actions prioritaires — Court terme

IV.2 Actions structurelles — Moyen terme

IV.3 Indicateurs de suivi recommandés

V. CONCLUSION

BIBLIOGRAPHIE / WEBOGRAPHIE

ANNEXES

I. INTRODUCTION

I.1 Contexte et problématique

Le turnover du personnel constitue l'un des défis les plus critiques auxquels font face les organisations, en particulier dans les secteurs à forte valeur intellectuelle tels que l'audit interne et le conseil. Au Maroc, les cabinets d'audit et les directions d'audit interne des grandes entreprises sont confrontés à un taux de rotation élevé parmi leurs auditeurs, générant des coûts directs et indirects considérables.

Il a été estimé que le remplacement d'un auditeur peut représenter entre **50 % et 200 %** de son salaire annuel, en incluant les coûts de recrutement, de formation et de montée en compétences. À ces charges financières s'ajoutent des conséquences organisationnelles non négligeables : perte de la mémoire institutionnelle, perturbation des missions en cours, dégradation de la qualité des livrables et effet de démotivation sur les équipes restantes.

> **Problématique centrale :** Comment prédire, à l'aide de l'Intelligence Artificielle et du Machine Learning, la probabilité qu'un auditeur interne quitte l'organisation dans les 12 mois à venir, afin de permettre une intervention préventive ciblée ?

I.2 Objectifs du projet

Ce projet poursuit trois objectifs complémentaires et articulés :

- **Objectif technique :** Construire un modèle prédictif capable de classifier un auditeur comme « à risque de départ » ou « stable », à partir de variables RH historiques.
- **Objectif analytique :** Identifier et hiérarchiser les facteurs les plus déterminants du turnover dans le contexte spécifique de l'audit interne.
- **Objectif métier :** Formuler des recommandations concrètes et actionnables à destination des cabinets d'audit et directions des ressources humaines marocains.

I.3 Méthodologie adoptée

Le Machine Learning est capable d'apprendre des patterns complexes et non linéaires à partir de données historiques pour produire des prédictions individualisées et actionnables. La démarche méthodologique retenue s'articule en six étapes successives :

1. Collecte et présentation du dataset (IBM HR Analytics Employee Attrition Dataset — Kaggle)
2. Exploration et Analyse des Données (EDA) : distributions, corrélations, détection d'anomalies
3. Prétraitement : nettoyage, encodage, rééquilibrage des classes par SMOTE, normalisation
4. Modélisation comparative : entraînement de cinq algorithmes de classification avec validation croisée
5. Évaluation des performances : AUC-ROC, F1-Score, Recall, matrices de confusion, courbes ROC
6. Interprétabilité des prédictions via les valeurs SHAP, et formulation de recommandations métier

II. DÉVELOPPEMENT

II.1 Présentation du dataset IBM HR Analytics

Le projet s'appuie sur l'**IBM HR Analytics Employee Attrition & Performance Dataset**, disponible publiquement sur la plateforme Kaggle. Il s'agit d'un jeu de données de référence dans le domaine du People Analytics, fréquemment utilisé dans la littérature académique et les compétitions data science.

Source : https://www.kaggle.com/datasets/pavansubhasht/ibm-hr-analytics-attrition-dataset

Caractéristique	Valeur
Nombre d'observations	1 470 employés
Nombre de variables	35 colonnes
Type de tâche	Classification binaire supervisée
Variable cible	Attrition (Yes = départ / No = reste)
Valeurs manquantes	Aucune

Tableau 1 : Caractéristiques générales du dataset IBM HR Analytics

DISTRIBUTION DE LA VARIABLE CIBLE

Classe	Effectif	Proportion
Non — Reste (0)	1 233	83,9 %
Oui — Départ (1)	237	16,1 %

Tableau 2 : Distribution de la variable cible Attrition

VARIABLES CLÉS RETENUES

- **Variables démographiques :** Âge, Genre, Situation matrimoniale, Distance domicile-travail
- **Variables professionnelles :** Fonction, Niveau hiérarchique, Département, Ancienneté, Expérience totale, Nombre d'employeurs précédents, Fréquence des déplacements
- **Variables de satisfaction et rémunération :** Salaire mensuel, Satisfaction au travail (1–4), Équilibre vie pro/perso (1–4), Heures supplémentaires (OverTime), Augmentation salariale, Options sur actions

II.2 Exploration et Analyse des Données (EDA)

ÉTAPE 1 — VÉRIFICATION DES VALEURS MANQUANTES

Code Python

```
# Verification des valeurs manquantes
valeurs_manquantes = df.isnull().sum()
if valeurs_manquantes.sum() == 0:
    print('Aucune valeur manquante detectee.')
```

Output

```
Valeurs manquantes par colonne :
Aucune valeur manquante detectee.
```

DISTRIBUTION DE LA VARIABLE CIBLE ATTRITION

Code Python

```
# Distribution de la variable cible : Attrition
counts = df['Attrition'].value_counts()
pcts = df['Attrition'].value_counts(normalize=True) * 100
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
# Diagramme en barres + diagramme circulaire
plt.tight_layout()
plt.show()
```

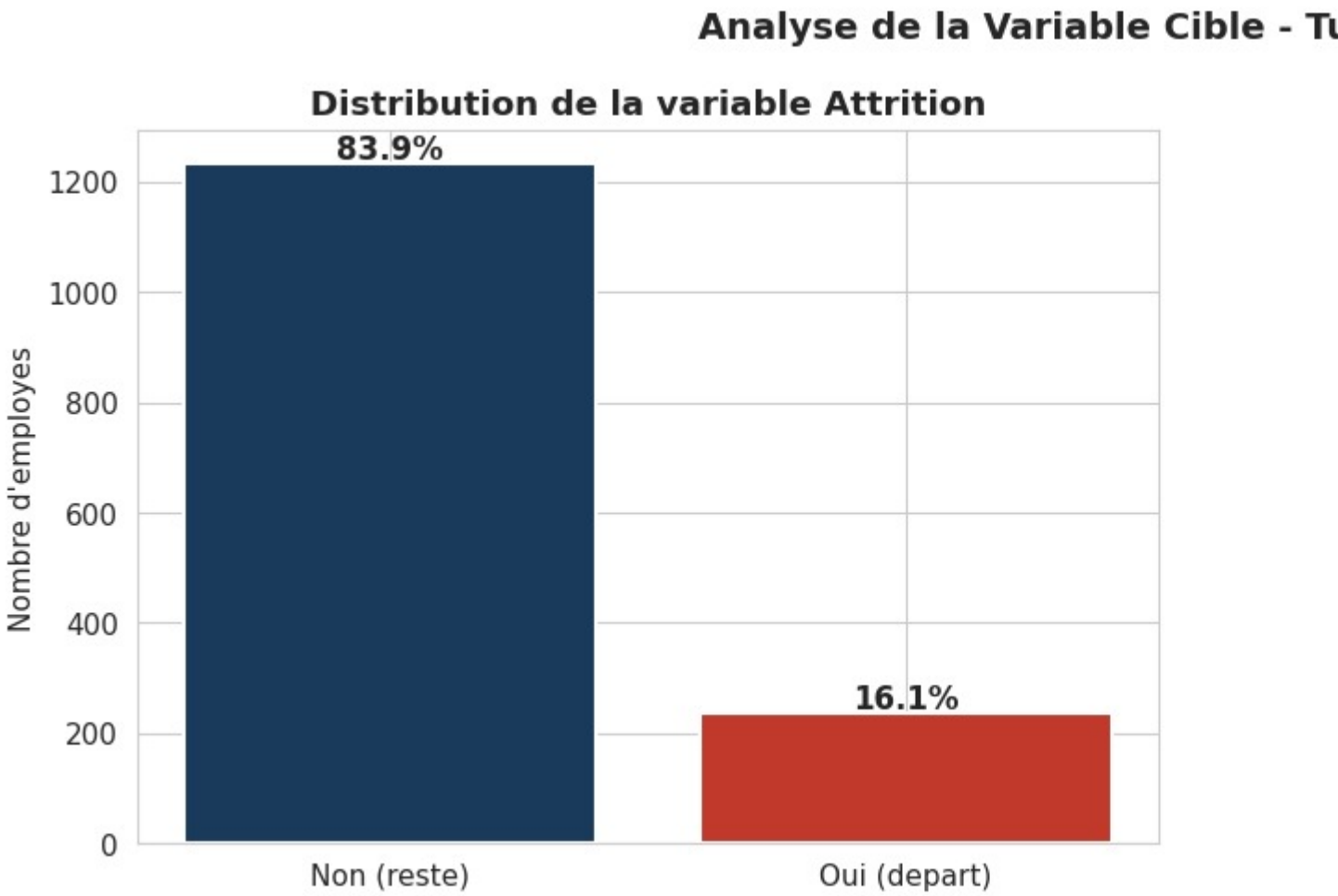


Figure 1 : Distribution de la variable cible Attrition — 83,9 % restent, 16,1 % partent

BOXPLOTS DES VARIABLES NUMÉRIQUES STRATIFIÉS PAR ATTRITION

Code Python

```
variables_num = ['Age', 'MonthlyIncome', 'YearsAtCompany',
                 'DistanceFromHome', 'TotalWorkingYears',
                 'YearsWithCurrManager', 'NumCompaniesWorked', 'JobLevel']
fig, axes = plt.subplots(2, 4, figsize=(18, 9))
for i, var in enumerate(variables_num):
    df.boxplot(column=var, by='Attrition', ax=axes.flatten()[i], ...)
plt.show()
```

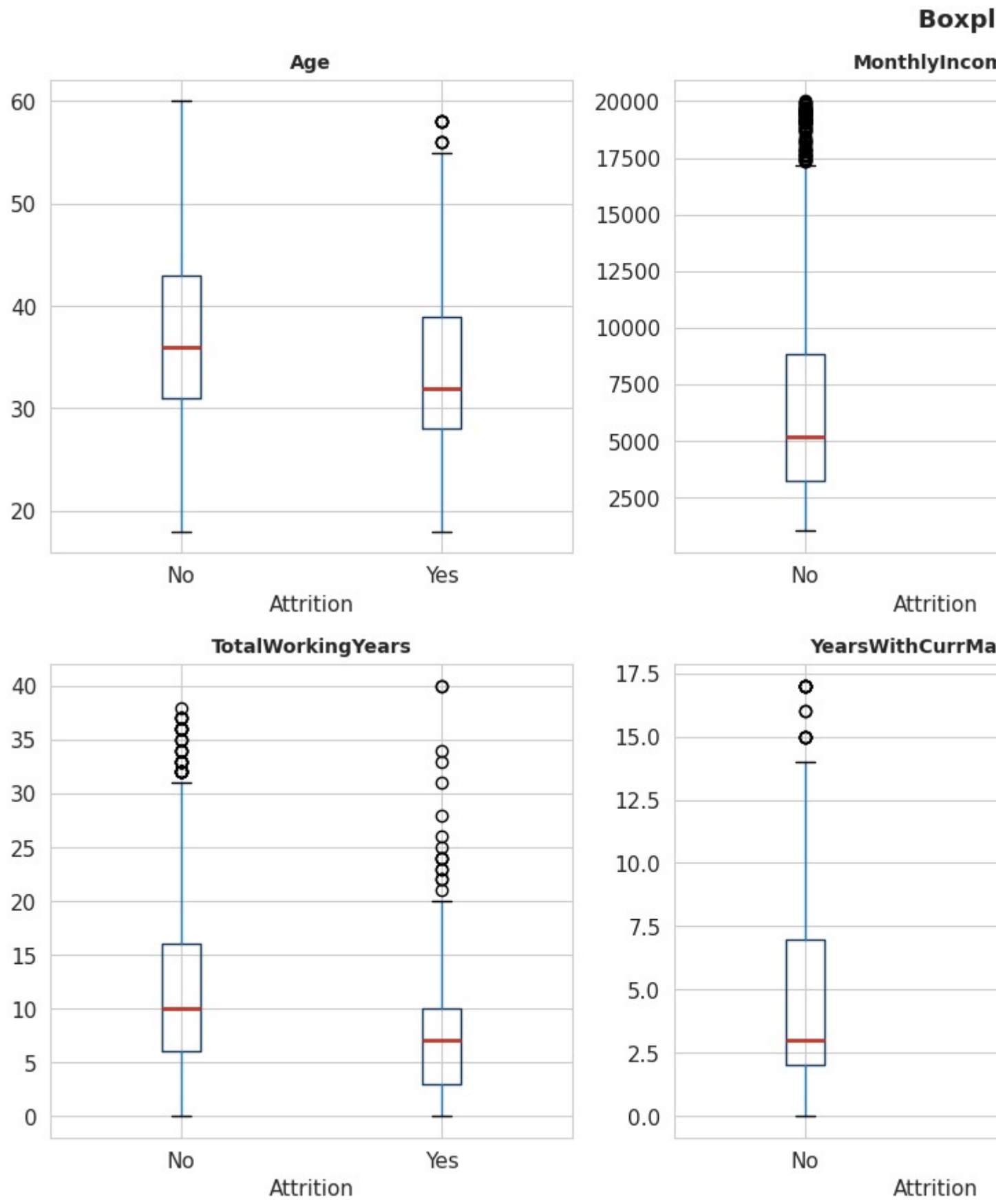


Figure 2 : Boxplots des variables numériques stratifiés par Attrition

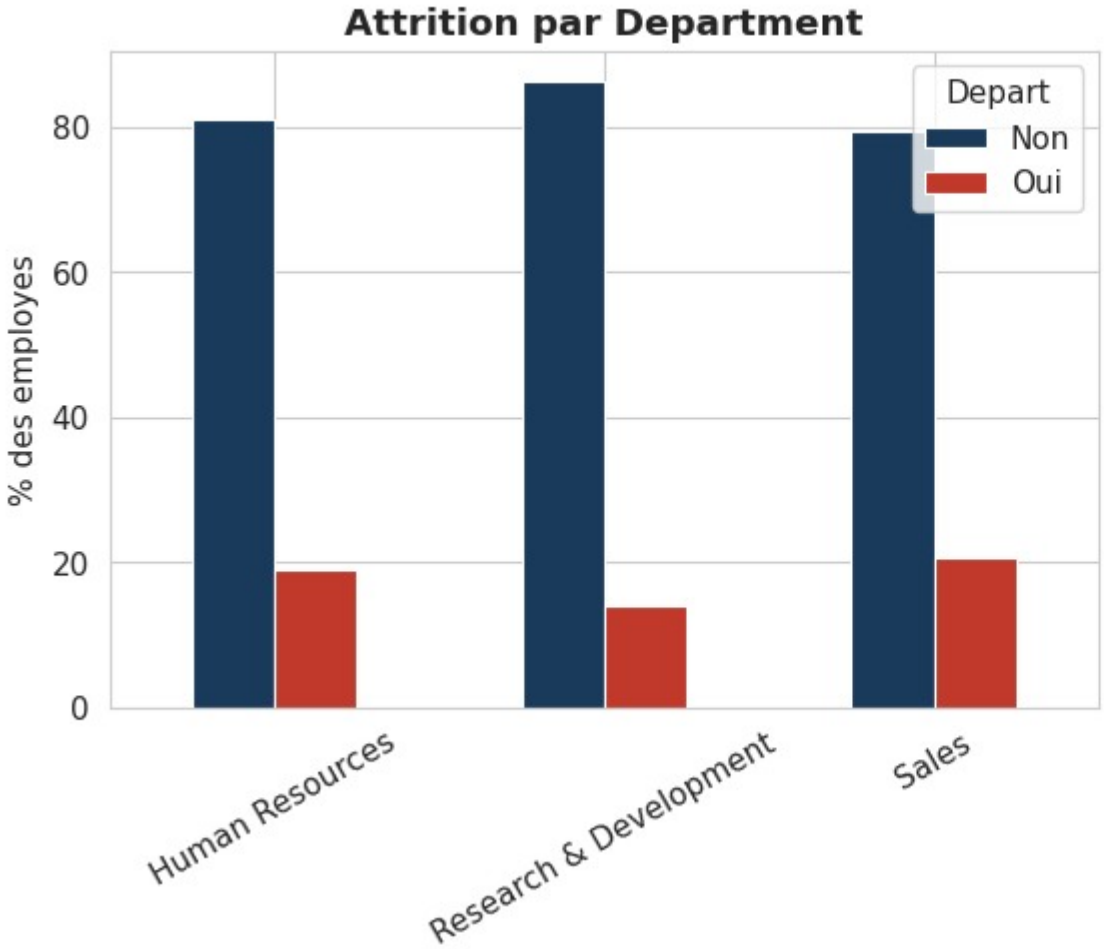
Variable	Observation clé
Age	Les employés de moins de 35 ans présentent un taux de départ plus élevé
MonthlyIncome	Les auditeurs quittant l'entreprise ont des salaires significativement plus bas
YearsAtCompany	Le turnover est concentré chez les employés avec moins de 5 ans d'ancienneté
DistanceFromHome	Les distances importantes sont associées à un risque de départ accru
TotalWorkingYears	Les profils juniors (peu d'expérience totale) sont plus volatils
JobLevel	Les niveaux hiérarchiques inférieurs enregistrent davantage de départs
YearsWithCurrManager	Un faible tenure avec le manager actuel corrèle avec le départ

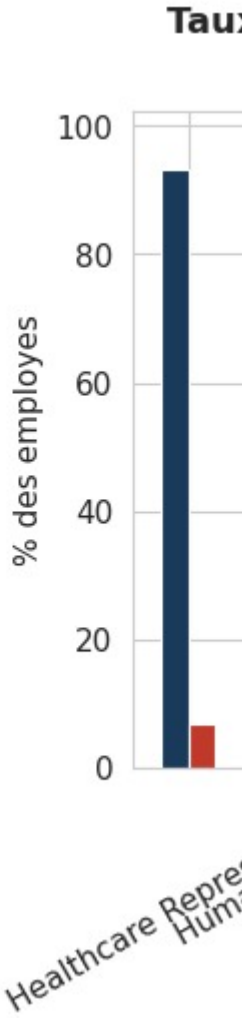
Tableau 3 : Principaux résultats de l'analyse univariée des variables numériques

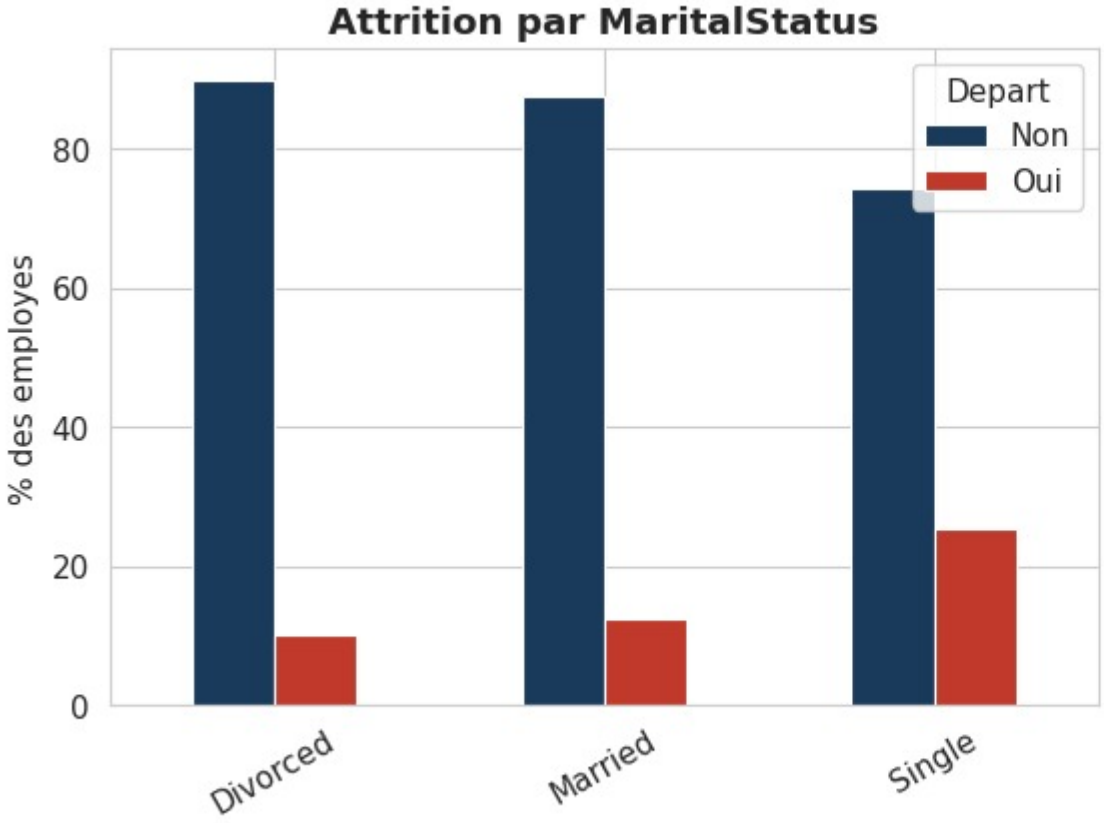
ANALYSE UNIVARIÉE — VARIABLES CATÉGORIELLES

Code Python

```
variables_cat = ['Department', 'JobRole', 'OverTime', 'MaritalStatus', 'BusinessTravel']
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
for i, var in enumerate(variables_cat):
    ct = pd.crosstab(df[var], df['Attrition'], normalize='index') * 100
    ct.plot(kind='bar', ax=axes.flatten()[i], color=['#1a5c2a', '#c45000'])
plt.show()
```


Attrition par Department
Depart
Non
Oui
% des employes
0
20
40
60
80
Human Resources
Research & Development
Sales


Tau
% des employes
0
20
40
60
80
100
Healthcare Repre
Huma


Attrition par MaritalStatus
Depart
Non
Oui
% des employes
0
20
40
60
80
Divorced
Married
Single

% des employes
0
20
40
60
80

Figure 3 : Taux d'attrition par variable catégorielle (Department, JobRole, OverTime, MaritalStatus, BusinessTravel)

- **OverTime :** Les employés effectuant régulièrement des heures supplémentaires affichent un taux de départ nettement supérieur (~30 % contre ~10 %). Il s'agit du **signal prédictif le plus puissant** du dataset.
- **BusinessTravel :** Le groupe Travel_Frequently enregistre le taux d'attrition le plus élevé.
- **MaritalStatus :** Les célibataires présentent une mobilité supérieure et sont davantage susceptibles de saisir des opportunités externes.

MATRICE DE CORRÉLATION

Code Python

```
df_num = df.select_dtypes(include=[np.number]).copy()
df_num['Attrition_num'] = (df['Attrition'] == 'Yes').astype(int)
corr = df_num.corr()
sns.heatmap(corr, mask=masque, annot=True, fmt='.2f', cmap='RdBu_r', center=0)
plt.title('Matrice de Correlation', fontsize=14, fontweight='bold')
plt.show()
```

Matrice

	Age	DailyRate	DistanceFromHome	Education	EmployeeCount	EmployeeNumber	EnvironmentSatisfaction	HourlyRate	JobInvolvement	JobLevel	JobSatisfaction	MonthlyIncome	MonthlyRate	NumCompaniesWorked	PercentSalaryHike	PerformanceRating	RelationshipSatisfaction	StandardHours	StockOptionLevel	TotalWorkingYears	TrainingTimesLastYear	WorkLifeBalance	YearsAtCompany	YearsInCurrentRole	YearsSinceLastPromotion	YearsWithCurrManager	Attrition_num
Age																											
DailyRate	0.01																										
DistanceFromHome	-0.00	-0.00																									
Education	0.21	-0.02	0.02																								
EmployeeCount																											
EmployeeNumber	-0.01	-0.05	0.03	0.04																							
EnvironmentSatisfaction	0.01	0.02	-0.02	-0.03		0.02																					
HourlyRate	0.02	0.02	0.03	0.02		0.04	-0.05																				
JobInvolvement	0.03	0.05	0.01	0.04		-0.01	-0.01	0.04																			
JobLevel	0.51	0.00	0.01	0.10		-0.02	0.00	-0.03	-0.01																		
JobSatisfaction	-0.00	0.03	-0.00	-0.01		-0.05	-0.01	-0.07	-0.02	-0.00																	
MonthlyIncome	0.50	0.01	-0.02	0.09		-0.01	-0.01	-0.02	-0.02	0.95	-0.01																
MonthlyRate	0.03	-0.03	0.03	-0.03		0.01	0.04	-0.02	-0.02	0.04	0.00	0.03															
NumCompaniesWorked	0.30	0.04	-0.03	0.13		-0.00	0.01	0.02	0.02	0.14	-0.06	0.15	0.02														
PercentSalaryHike	0.00	0.02	0.04	-0.01		-0.01	-0.03	-0.01	-0.02	-0.03	0.02	-0.03	-0.01														
PerformanceRating	0.00	0.00	0.03	-0.02		-0.02	-0.03	-0.00	-0.03	-0.02	0.00	-0.02	-0.01														
RelationshipSatisfaction	0.05	0.01	0.01	-0.01		-0.07	0.01	0.00	0.03	0.02	-0.01	0.03	-0.00														
StandardHours																											
StockOptionLevel	0.04	0.04	0.04	0.02		0.06	0.00	0.05	0.02	0.01	0.01	0.01	-0.03														
TotalWorkingYears	0.68	0.01	0.00	0.15		-0.01	-0.00	-0.00	-0.01	0.78	-0.02	0.77	0.03														
TrainingTimesLastYear	-0.02	0.00	-0.04	-0.03		0.02	-0.02	-0.01	-0.02	-0.02	-0.01	-0.02	0.00														
WorkLifeBalance	-0.02	-0.04	-0.03	0.01		0.01	0.03	-0.00	-0.01	0.04	-0.02	0.03	0.01														
YearsAtCompany	0.31	-0.03	0.01	0.07		-0.01	0.00	-0.02	-0.02	0.53	-0.00	0.51	-0.02														
YearsInCurrentRole	0.21	0.01	0.02	0.06		-0.01	0.02	-0.02	0.01	0.39	-0.00	0.36	-0.01														
YearsSinceLastPromotion	0.22	-0.03	0.01	0.05		-0.01	0.02	-0.03	-0.02	0.35	-0.02	0.34	0.00														
YearsWithCurrManager	0.20	-0.03	0.01	0.07		-0.01	-0.00	-0.02	0.03	0.38	-0.03	0.34	-0.04														
Attrition_num	-0.16	-0.06	0.08	-0.03		-0.01	-0.10	-0.01	-0.13	-0.17	-0.10	-0.16	0.02														

Figure 4 : Matrice de corrélation — visualisation des liaisons entre variables

TOP 10 VARIABLES CORRÉLÉES AVEC L'ATTRITION

Code Python

```
top_corr = (corr['Attrition_num'].drop('Attrition_num')
            .abs().sort_values(ascending=False).head(10))
plt.barh(top_corr.index[::-1], top_corr.values[::-1],
         color=plt.cm.Blues(np.linspace(0.4, 0.9, 10)))
plt.title('Top 10 Variables les Plus Correlees avec le Depart')
plt.show()
```

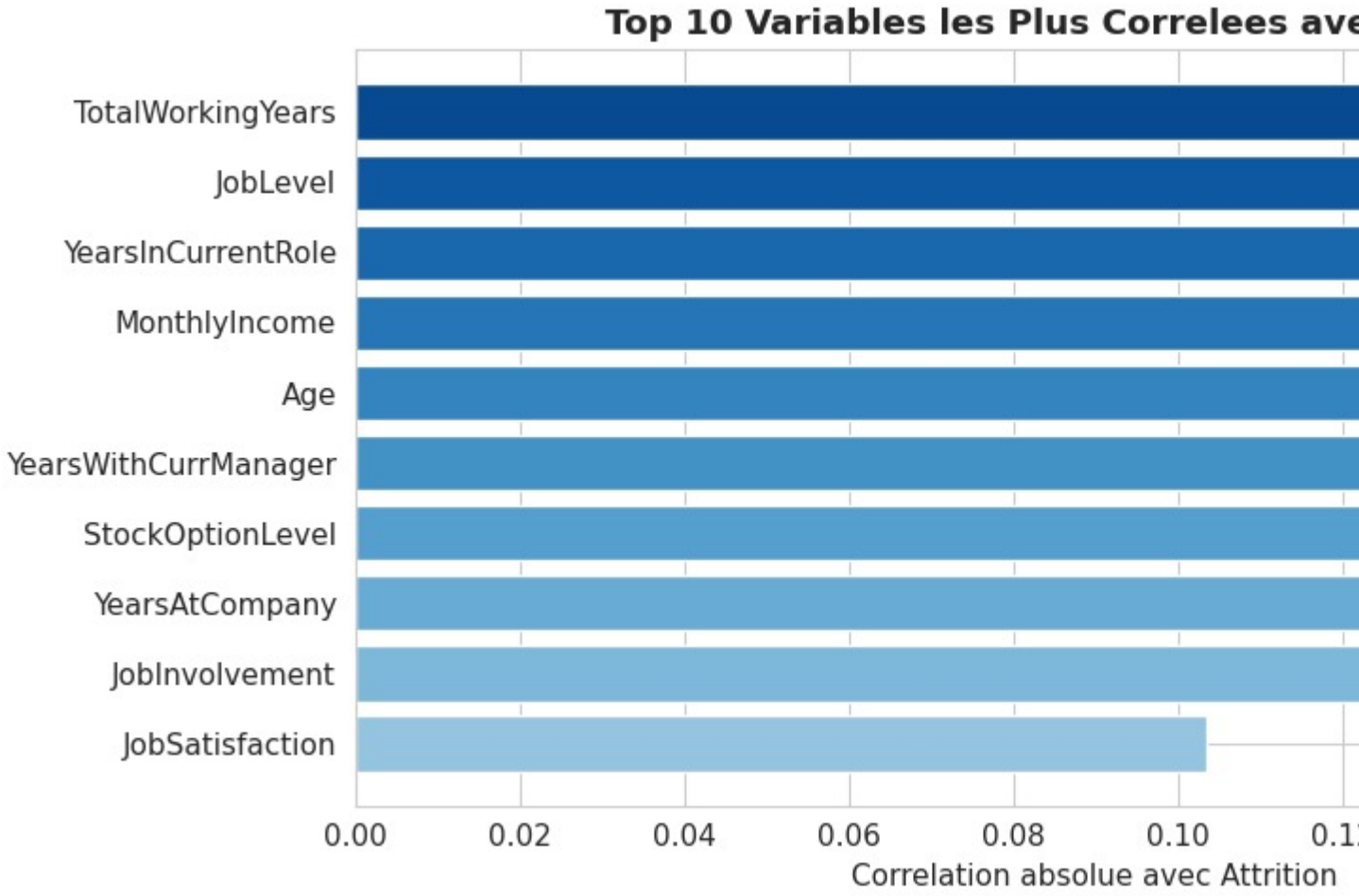


Figure 5 : Top 10 variables les plus corrélées avec l'Attrition

Rang	Variable	Corrélation avec Attrition
1	OverTime	Forte positive
2	MonthlyIncome	Forte négative
3	Age	Modérée négative
4	TotalWorkingYears	Modérée négative
5	JobLevel	Modérée négative
6	YearsInCurrentRole	Modérée négative
7	YearsWithCurrManager	Modérée négative
8	MaritalStatus	Modérée positive
9	DistanceFromHome	Faible positive
10	NumCompaniesWorked	Faible positive

Tableau 4 : Classement des variables selon leur corrélation avec l'Attrition

II.3 Prétraitement des Données

Le pipeline de prétraitement a été structuré en six étapes successives, appliquées dans un ordre méthodologiquement rigoureux visant à éviter toute fuite de données (*data leakage*) entre les ensembles d'entraînement et de test.

ÉTAPE 1 — SUPPRESSION DES COLONNES NON INFORMATIVES

Code Python

```
df_prep = df.copy()
df_prep['Attrition'] = (df_prep['Attrition'] == 'Yes').astype(int)
colonnes_a_supprimer = ['EmployeeCount', 'Over18', 'StandardHours', 'EmployeeNumber']
df_prep.drop(columns=colonnes_a_supprimer, inplace=True)
print(f'Colonnes supprimees : {colonnes_a_supprimer}')
print(f'Dimensions apres suppression : {df_prep.shape}')
```

Output

```
Colonnes supprimees : ['EmployeeCount', 'Over18', 'StandardHours', 'EmployeeNumber']
Dimensions apres suppression : (1470, 31)
```

ÉTAPE 2 — ENCODAGE DES VARIABLES CATÉGORIELLES (LABELENCODER)

Code Python

```
colonnes_cat = df_prep.select_dtypes(include=['object']).columns.tolist()
le = LabelEncoder()
for col in colonnes_cat:
    df_prep[col] = le.fit_transform(df_prep[col])
```

Output

```
Variables categorielles encodees : ['BusinessTravel', 'Department', 'EducationField',
                                    'Gender', 'JobRole', 'MaritalStatus', 'OverTime']
7 variables encodees avec succes.
```

ÉTAPE 3 — SÉPARATION FEATURES / VARIABLE CIBLE

Code Python

```
X = df_prep.drop('Attrition', axis=1)
y = df_prep['Attrition']
```

Output

```
Dimensions X : (1470, 30)
Dimensions y : (1470,)
Distribution y -> Reste (0): 1233 | Depart (1): 237
```

ÉTAPE 4 — DIVISION TRAIN/TEST (80/20) AVEC STRATIFICATION

Code Python

```python
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Output

```
Ensemble entrainement : 1176 lignes
Ensemble test : 294 lignes
```

ÉTAPE 5 — RÉÉQUILIBRAGE SMOTE

Code Python

```python
sm = SMOTE(random_state=42)
X_train_sm, y_train_sm = sm.fit_resample(X_train, y_train)
print(f'Avant SMOTE -> Reste: {(y_train == 0).sum()} | Depart: {(y_train == 1).sum()}')
print(f'Apres SMOTE -> Reste: {(y_train_sm == 0).sum()} | Depart: {(y_train_sm == 1).sum()}')
```

Output

```
Avant SMOTE -> Reste: 986 | Depart: 190
Apres SMOTE -> Reste: 986 | Depart: 986
Classes equilibrees avec succes.
```

ÉTAPE 6 — NORMALISATION STANDARDSCALER

Code Python

```python
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_sm)
X_test_scaled = scaler.transform(X_test)
```

Output

```
Normalisation appliquee (StandardScaler).
```

Ensemble	Avant SMOTE — Cl. 0	Avant SMOTE — Cl. 1	Après SMOTE
Entraînement (80 %)	986	190	986 / 986
Test (20 %)	249	45	Non modifié

Tableau 5 : Impact du rééquilibrage SMOTE sur la distribution des classes

II.4 Modèles de Machine Learning testés

Code Python

```python
modeles = {
    'Logistic Regression' : LogisticRegression(max_iter=1000, random_state=42),
    'Random Forest'       : RandomForestClassifier(n_estimators=200, random_state=42),
    'XGBoost'             : XGBClassifier(eval_metric='logloss', random_state=42),
    'LightGBM'            : LGBMClassifier(random_state=42, verbose=-1),
    'Gradient Boosting'   : GradientBoostingClassifier(n_estimators=200, random_state=42),
}
```

Output

```
5 modeles definis.
```

#	Modèle	Famille	Hyperparamètres principaux
1	Logistic Regression	Linéaire	max_iter=1000, random_state=42
2	Random Forest	Ensemble — Bagging	n_estimators=200, random_state=42
3	XGBoost	Ensemble — Boosting	eval_metric='logloss', random_state=42
4	LightGBM	Ensemble — Boosting	random_state=42, verbose=-1
5	Gradient Boosting	Ensemble — Boosting	n_estimators=200, random_state=42

Tableau 6 : Présentation des cinq algorithmes de classification testés

ENTRAÎNEMENT ET VALIDATION CROISÉE (K=5)

Code Python

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
for nom, modele in modeles.items():
    modele.fit(X_train_scaled, y_train_sm)
    y_pred = modele.predict(X_test_scaled)
    y_proba = modele.predict_proba(X_test_scaled)[:, 1]
    scores_cv = cross_val_score(modele, X_train_scaled, y_train_sm,
                                cv=cv, scoring='roc_auc')
```

Output

```
Logistic Regression | AUC-ROC: 0.7182 | F1: 0.4486 | Recall: 0.5106 | CV: 0.9139
Random Forest       | AUC-ROC: 0.7346 | F1: 0.3059 | Recall: 0.2766 | CV: 0.9684
XGBoost             | AUC-ROC: 0.7411 | F1: 0.3855 | Recall: 0.3404 | CV: 0.9698
LightGBM            | AUC-ROC: 0.7521 | F1: 0.3765 | Recall: 0.3404 | CV: 0.9705
Gradient Boosting   | AUC-ROC: 0.7633 | F1: 0.4694 | Recall: 0.4894 | CV: 0.9617
```

II.5 Interprétabilité avec SHAP

SHAP (SHapley Additive exPlanations) est une technique d'interprétabilité fondée sur la théorie des jeux coopératifs. Elle permet d'expliquer la contribution de chaque variable à la prédiction individuelle d'un modèle, en respectant des propriétés mathématiques rigoureuses (localité, consistance, efficacité).

Code Python

```
explainer = shap.TreeExplainer(meilleur_modele)
shap_vals = explainer.shap_values(X_test_df)
print('Valeurs SHAP calculees avec succes.')
```

Output

```
Valeurs SHAP calculees avec succes.
```

Seuil de probabilité	Action recommandée
> 70 % — Risque élevé	Entretien de rétention URGENT
40 % – 70 % — Risque modéré	Suivi rapproché + plan de développement
< 40 % — Risque faible	Aucune action urgente nécessaire

Tableau 7 : Grille d'intervention basée sur la probabilité de départ prédite

III. RÉSULTATS ET DISCUSSION

III.1 Tableau comparatif des performances

Code Python

```
tableau_metriques = pd.DataFrame({
    nom: {'Accuracy': ..., 'Precision': ..., 'Recall': ...,
          'F1-Score': ..., 'AUC-ROC': ..., 'CV Mean': ...}
    for nom, r in resultats.items()}
).T.sort_values('AUC-ROC', ascending=False)
print(tableau_metriques)
```

Output

```
                   Accuracy  Precision  Recall  F1-Score  AUC-ROC  CV Mean
Gradient Boosting    0.8231     0.4510  0.4894    0.4694   0.7633   0.9617
LightGBM             0.8197     0.4211  0.3404    0.3765   0.7521   0.9705
XGBoost              0.8265     0.4444  0.3404    0.3855   0.7411   0.9698
Random Forest        0.7993     0.3421  0.2766    0.3059   0.7346   0.9684
Logistic Regression  0.7993     0.4000  0.5106    0.4486   0.7182   0.9139
```

Modèle	Accuracy	Precision	Recall	F1-Score	AUC-ROC	CV Mean
⭐ **Gradient Boosting**	**0.8231**	**0.4510**	**0.4894**	**0.4694**	**0.7633**	**0.9617**
LightGBM	0.8197	0.4211	0.3404	0.3765	0.7521	0.9705
XGBoost	0.8265	0.4444	0.3404	0.3855	0.7411	0.9698
Random Forest	0.7993	0.3421	0.2766	0.3059	0.7346	0.9684
Logistic Regression	0.7993	0.4000	0.5106	0.4486	0.7182	0.9139

Tableau 8 : Performances comparatives des cinq algorithmes (⭐ = meilleur modèle)

GRAPHIQUE COMPARATIF DES MÉTRIQUES

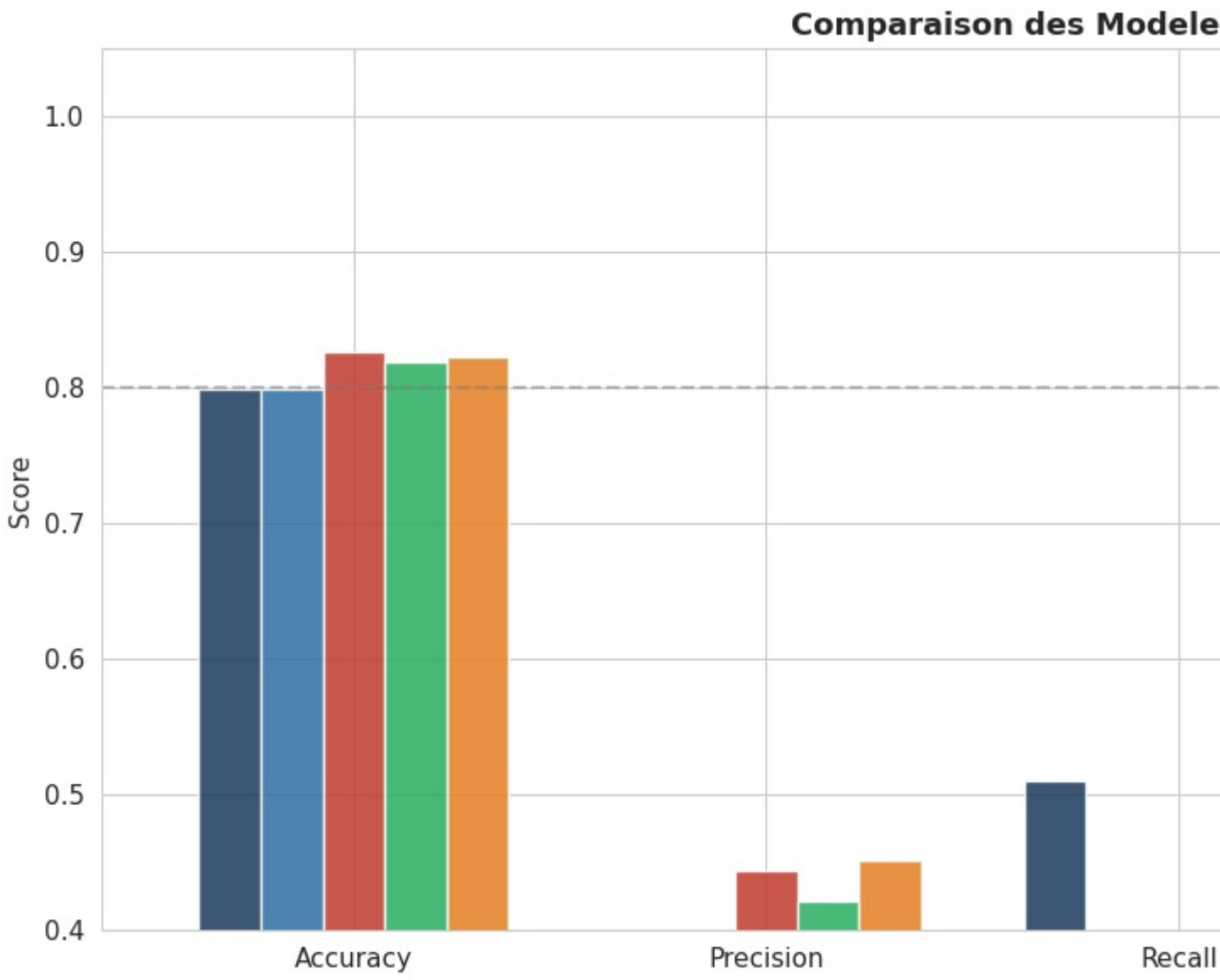


Figure 6 : Comparaison des performances des cinq algorithmes de classification

COURBES ROC COMPARATIVES

Code Python

```
plt.figure(figsize=(10, 7))
for i, (nom, r) in enumerate(resultats.items()):
    plt.plot(r['fpr'], r['tpr'], label=f'{nom} (AUC={r["auc_roc"]:.4f})')
plt.plot([0, 1], [0, 1], 'k--', label='Aleatoire (AUC=0.50)')
plt.show()
```

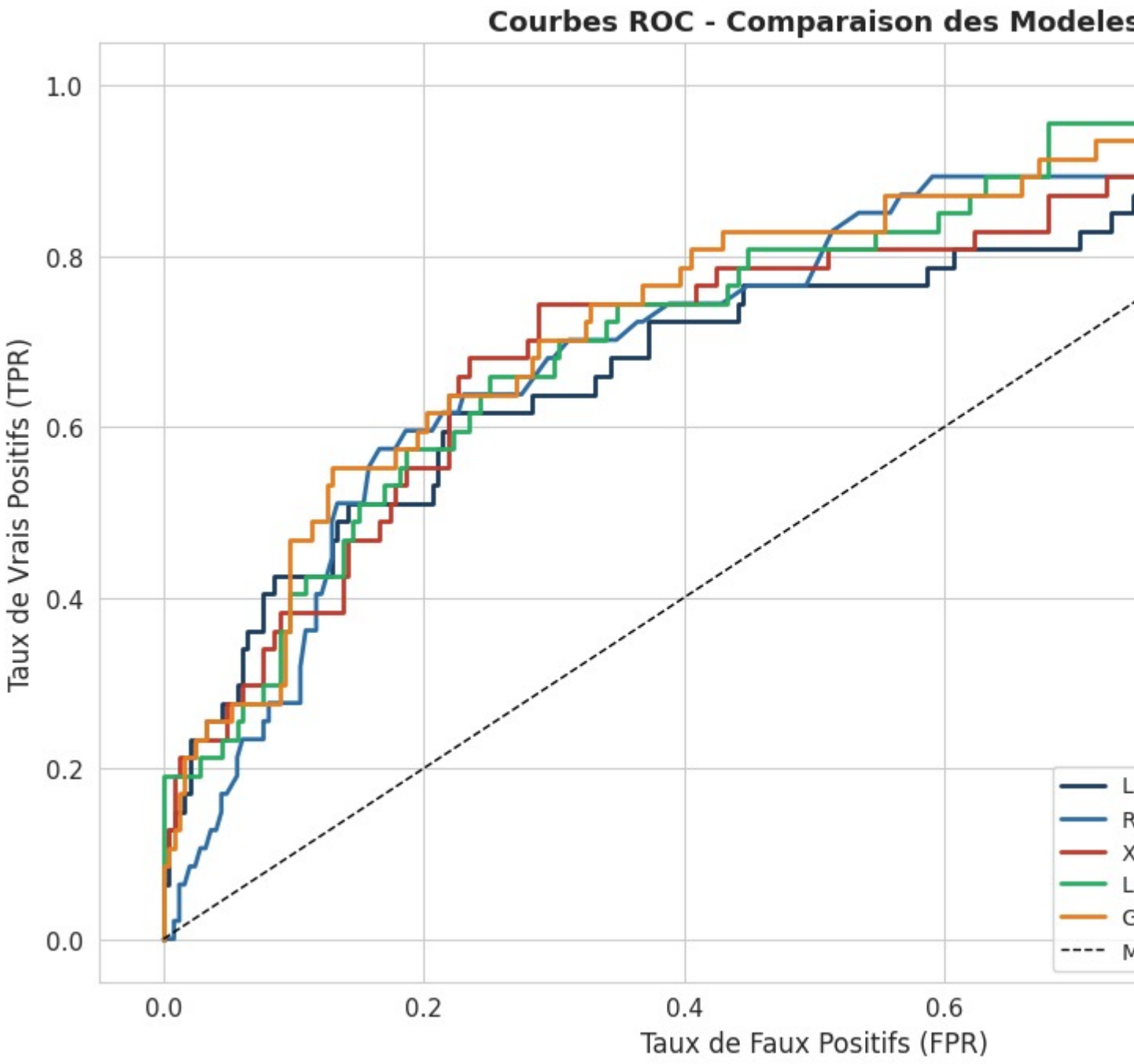


Figure 7 : Courbes ROC comparatives des cinq modèles

MATRICE DE CONFUSION — MEILLEUR MODÈLE (GRADIENT BOOSTING)

Code Python

```
cm = confusion_matrix(y_test, meilleur['y_pred'])
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Reste', 'Depart'],
            yticklabels=['Reste', 'Depart'])
plt.title(f'Matrice de Confusion — {meilleur_nom}')
plt.show()
```

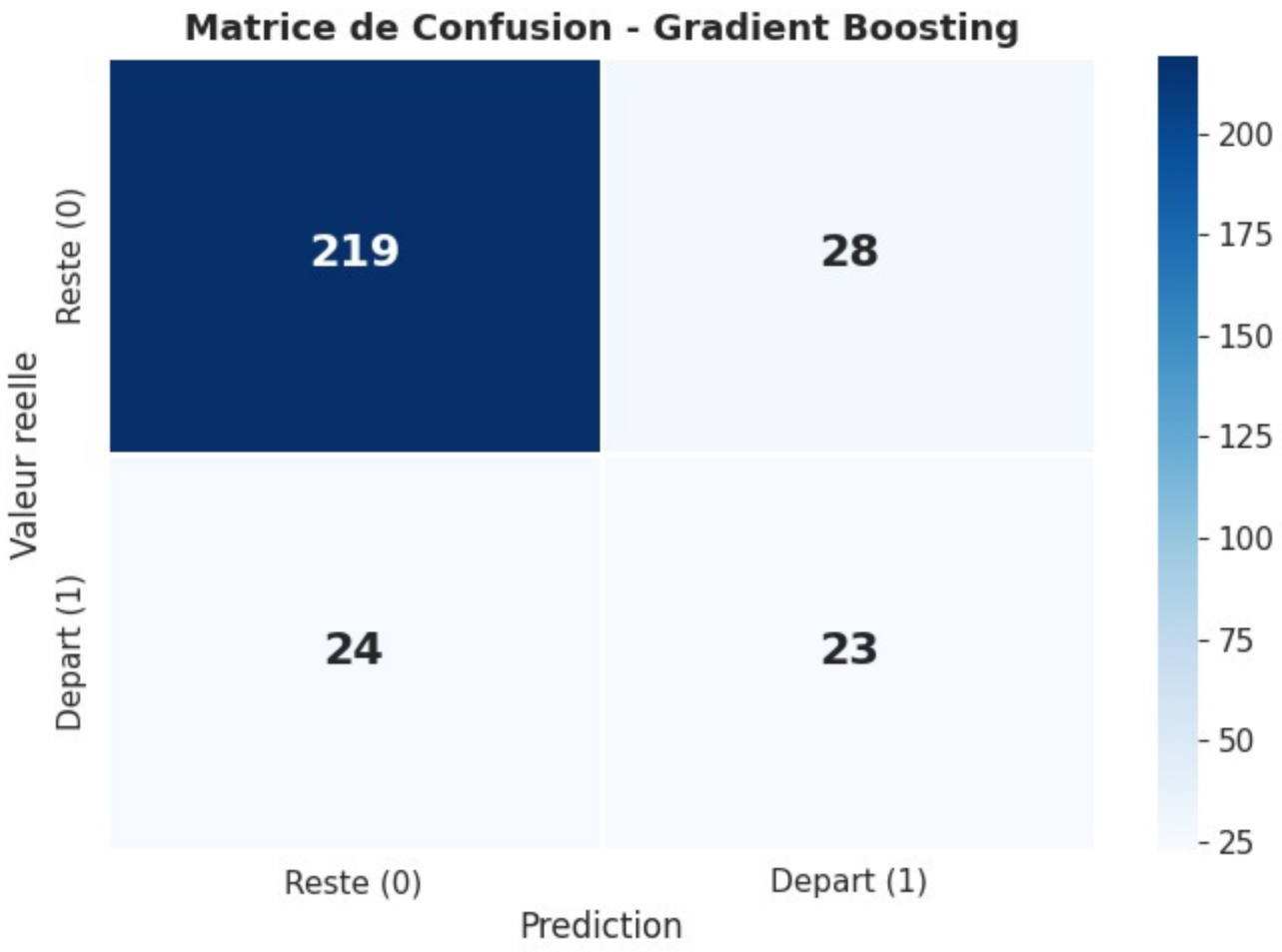


Figure 8 : Matrice de confusion du modèle Gradient Boosting (meilleur AUC-ROC)

SÉLECTION DU MEILLEUR MODÈLE

Code Python

```
meilleur_nom = max(resultats, key=lambda k: resultats[k]['auc_roc'])
print(f'Meilleur modele : {meilleur_nom}')
print(f'  AUC-ROC : {meilleur["auc_roc"]:.4f}')
print(f'  Recall  : {meilleur["recall"]:.4f}')
```

Output

```
Meilleur modele : Gradient Boosting
AUC-ROC    : 0.7633
Recall     : 0.4894
F1-Score   : 0.4694
Precision  : 0.4510
Accuracy   : 0.8231
```

🏆 **Meilleur Modèle : Gradient Boosting**

0.7633
AUC-ROC

0.8231
Accuracy

0.4894

Recall

0.4694
F1-Score

0.9617
CV Mean

III.2 Analyse SHAP et importance des variables

SHAP BAR PLOT — IMPORTANCE GLOBALE DES VARIABLES

Code Python

```
explainer = shap.TreeExplainer(meilleur_modele)
shap.summary_plot(sv, X_test_df, plot_type='bar', show=False)
plt.title('Importance Globale des Variables (SHAP)')
plt.show()
```

Importance Globale des Variables (SHAP) - Gradient
StockOptionLevel
JobSatisfaction
EnvironmentSatisfaction
JobInvolvement
BusinessTravel
YearsWithCurrManager
WorkLifeBalance
DistanceFromHome
OverTime
JobLevel
MonthlyIncome
Education
DailyRate
RelationshipSatisfaction
YearsSinceLastPromotion
TrainingTimesLastYear
EducationField
NumCompaniesWorked
Age
YearsInCurrentRole
0.0
0.2
0.4
0.6
0.8
mean(|SHAP value|) (average impact on model output m

Figure 9 : Importance globale des variables selon les valeurs SHAP — Gradient Boosting

SHAP BEESWARM PLOT — DIRECTION DES EFFETS

Code Python

```
shap.summary_plot(sv, X_test_df, feature_names=noms_features, show=False)
plt.title('SHAP - Impact des Variables sur la Prediction de Depart')
plt.show()
```

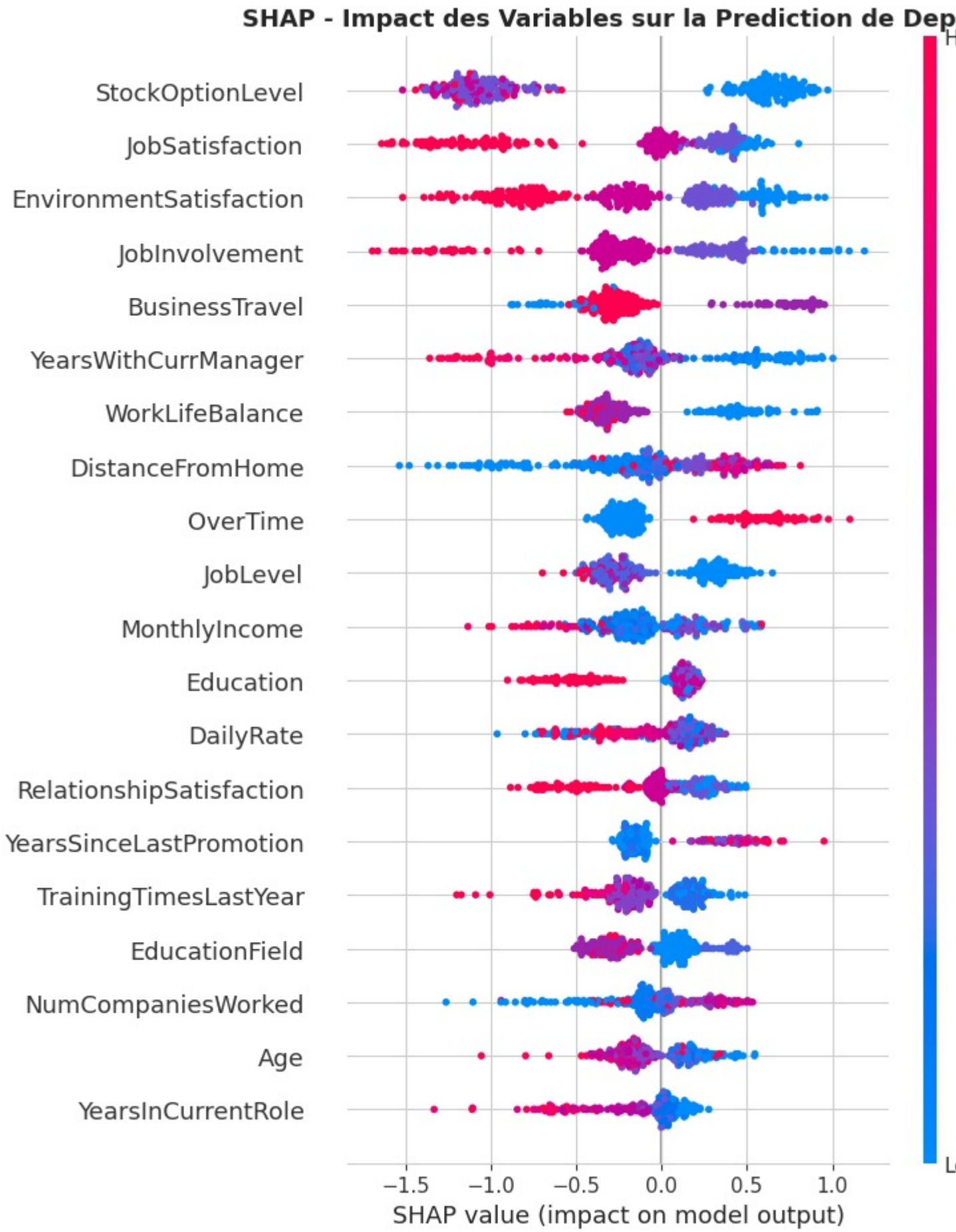

SHAP - Impact des Variables sur la Prediction de Dep
StockOptionLevel
JobSatisfaction
EnvironmentSatisfaction
JobInvolvement
BusinessTravel
YearsWithCurrManager
WorkLifeBalance
DistanceFromHome
OverTime
JobLevel
MonthlyIncome
Education
DailyRate
RelationshipSatisfaction
YearsSinceLastPromotion
TrainingTimesLastYear
EducationField
NumCompaniesWorked
Age
YearsInCurrentRole
−1.5
−1.0
−0.5
0.0
0.5
1.0
SHAP value (impact on model output)
H
L

Figure 10 : SHAP Beeswarm Plot — Direction des effets de chaque variable sur la prédiction

FEATURE IMPORTANCE INTERNE — TOP 15 VARIABLES

Code Python

```
importance = pd.Series(meilleur_modele.feature_importances_,
                       index=noms_features).sort_values(ascending=True).tail(15)
importance.plot(kind='barh', figsize=(10, 7))
plt.title('Top 15 Variables — Feature Importance (Gradient Boosting)')
plt.show()
```

Output

```
Top 5 facteurs predisant le depart :
1. StockOptionLevel      : 0.1736
2. JobSatisfaction       : 0.0890
3. EnvironmentSatisfaction : 0.0674
4. JobLevel              : 0.0570
5. YearsWithCurrManager  : 0.0545
```

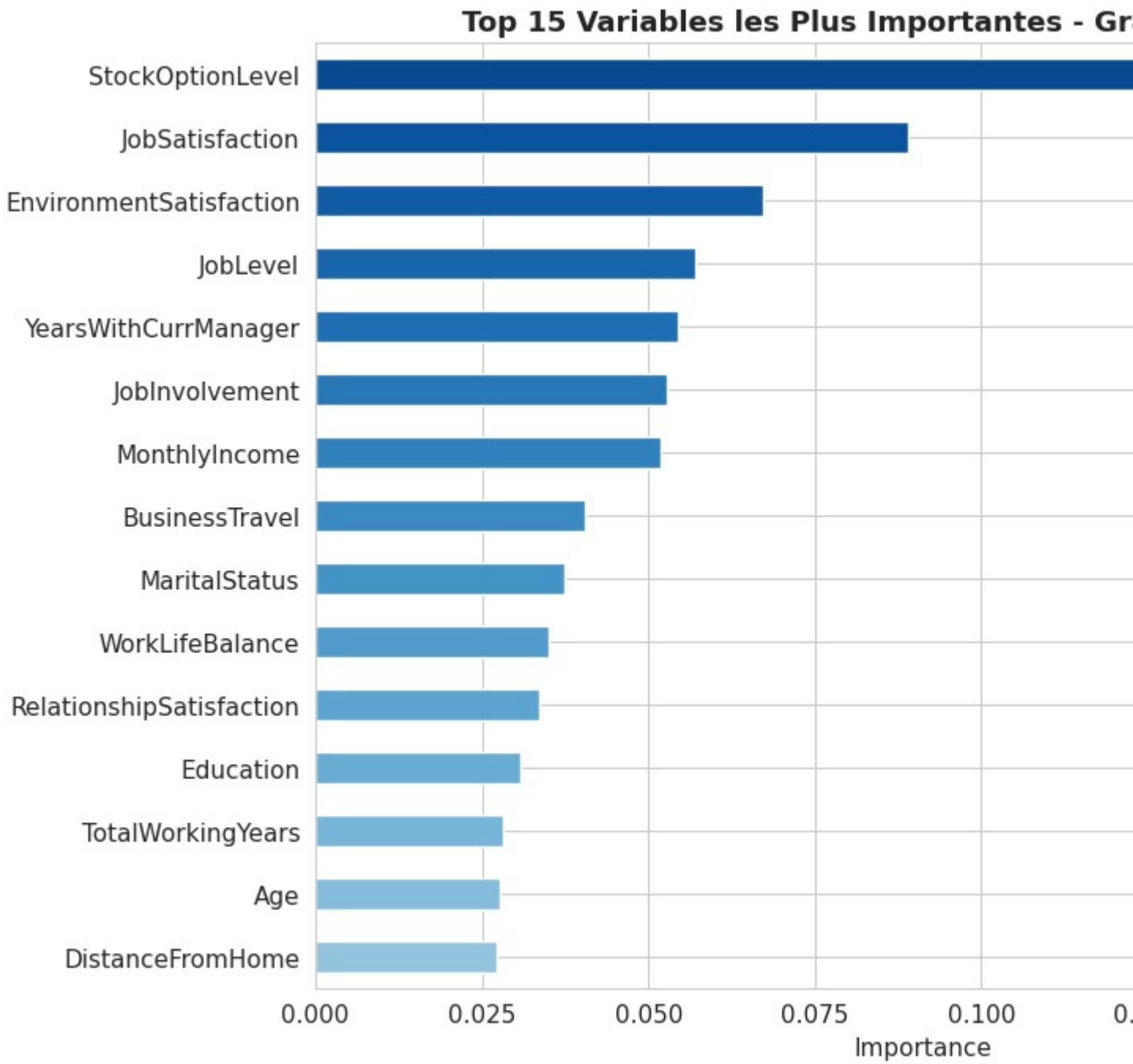


Figure 11 : Top 15 variables selon la feature importance interne — Gradient Boosting

Rang	Variable	Interprétation métier
1	**StockOptionLevel**	L'absence d'options sur actions réduit l'engagement financier
2	**JobSatisfaction**	Une faible satisfaction est un moteur de départ direct
3	**EnvironmentSatisfaction**	L'insatisfaction de l'environnement précède la démission
4	**JobLevel**	Les niveaux inférieurs manquent de perspectives d'évolution
5	**YearsWithCurrManager**	La qualité de la relation manager est un facteur de rétention clé

Tableau 9 : Top 5 variables selon l'importance SHAP — Gradient Boosting

III.3 Simulation individuelle

Code Python

```python
exemple = X_test_scaled[:1]
prediction = meilleur_modele.predict(exemple)[0]
probabilite = meilleur_modele.predict_proba(exemple)[0][1]
print(f'Probabilite de depart : {probabilite:.1%}')
print(f'Prediction : {"DEPART PROBABLE" if prediction == 1 else "STABLE"}')
```

Output

```
SIMULATION - Prediction pour un auditeur individuel
-----------------------------------------------
Probabilite de depart   : 64.7%
Prediction              : DEPART PROBABLE
Action recommandee      : Suivi rapproche et plan de developpement
```

> **Résultat de la simulation :** L'auditeur testé présente une probabilité de départ de **64,7 %**, le classant dans la zone de **risque modéré**. L'action recommandée est un suivi rapproché avec mise en place d'un plan de développement personnalisé.

III.4 Analyse critique et interprétation

La supériorité du **Gradient Boosting (AUC-ROC 0,7633)** confirme l'efficacité des méthodes de boosting pour les problèmes de classification déséquilibrée : la construction séquentielle d'arbres, chacun corrigeant les erreurs du précédent, s'avère particulièrement efficace pour apprendre les patterns caractéristiques des départs minoritaires.

La **Régression Logistique**, bien que moins performante sur l'AUC-ROC (0,7182), conserve néanmoins le meilleur Recall (0,5106), ce qui en fait un modèle intéressant pour maximiser la détection de tous les départs réels.

L'analyse SHAP révèle que les variables les plus prédictives — StockOptionLevel, JobSatisfaction, EnvironmentSatisfaction — sont précisément celles sur lesquelles les directions RH disposent des **leviers d'action les plus directs**.

III.5 Limites et axes d'amélioration

LIMITES IDENTIFIÉES

- Le dataset IBM est américain et peut ne pas refléter fidèlement le contexte des cabinets d'audit marocains
- L'absence d'un dataset propre aux auditeurs marocains limite la portée des conclusions
- Les hyperparamètres n'ont pas été optimisés par Grid Search ou Bayesian Optimization

AXES D'AMÉLIORATION

- Collecter des données RH réelles auprès de cabinets d'audit marocains (avec anonymisation)
- Optimiser les hyperparamètres via Optuna ou RandomizedSearchCV
- Explorer des modèles neuronaux adaptés aux données tabulaires (TabNet, AutoML)
- Développer un dashboard interactif de monitoring du risque de turnover en temps réel
- Intégrer des données temporelles pour un suivi longitudinal des indicateurs RH

IV. RECOMMANDATIONS MÉTIER

Sur la base des résultats analytiques et des insights SHAP, les recommandations suivantes sont formulées à l'attention des cabinets d'audit et directions d'audit interne marocains. Ces recommandations s'articulent en deux horizons temporels complémentaires.

IV.1 Actions prioritaires — Court terme

R1 — Revoir la politique d'options sur actions (StockOptionLevel)
Le niveau d'options sur actions est le facteur prédictif le plus puissant. Mettre en place ou renforcer les plans d'intéressement crée un engagement financier qui ancre l'auditeur dans l'organisation.

R2 — Améliorer la satisfaction au travail (JobSatisfaction)
Instaurer des enquêtes trimestrielles et des entretiens individuels réguliers. Un score inférieur à 2/4 doit déclencher automatiquement un entretien de rétention.

R3 — Entretiens de rétention proactifs
Utiliser le modèle pour identifier les auditeurs à risque élevé (probabilité > 70 %) et déclencher des entretiens avant que la décision de démission ne soit prise.

IV.2 Actions structurelles — Moyen terme

R4 — Améliorer l'environnement de travail (EnvironmentSatisfaction)
Espaces collaboratifs modernes, télétravail partiel, limitation des missions longue durée hors site, flexibilité des horaires.

R5 — Clarifier les plans de carrière (JobLevel)
Des trajectoires d'évolution transparentes et rapides doivent être proposées aux jeunes auditeurs (ancienneté < 5 ans).

R6 — Optimiser la relation manager–auditeur (YearsWithCurrManager)
Former les managers seniors à la détection précoce et à la gestion du risque de départ.

R7 — Renforcer les programmes de formation continue
Certifications CIA, CISA, normes IFRS, IA appliquée à l'audit : levier de rétention ET d'attractivité.

IV.3 Indicateurs de suivi recommandés

KPI	Cible	Fréquence
Taux de turnover global	< 15 %	Annuel
% d'auditeurs à risque élevé (modèle)	< 10 %	Trimestriel
Score JobSatisfaction moyen	≥ 3/4	Semestriel
Score EnvironmentSatisfaction moyen	≥ 3/4	Semestriel
% d'auditeurs avec plan de carrière formalisé	> 80 %	Annuel
% d'auditeurs avec stock options	> 50 %	Annuel

Tableau 10 : Tableau de bord des indicateurs de suivi RH recommandés

V. CONCLUSION

V.1 Synthèse des résultats

Ce projet a démontré la faisabilité et la valeur ajoutée du Machine Learning appliqué à la problématique RH du turnover des auditeurs internes. L'ensemble des étapes définies en introduction ont été menées à leur terme, depuis la préparation des données jusqu'à la formulation de recommandations actionnables.

#	Étape	Statut
1	Installation et importation des bibliothèques	✓ Complété
2	Chargement du dataset IBM HR Analytics	✓ Complété
3	Exploration (EDA) : distributions, corrélations	✓ Complété
4	Prétraitement : encodage, SMOTE, normalisation	✓ Complété
5	Modélisation : 5 modèles avec validation croisée	✓ Complété
6	Évaluation : AUC-ROC, F1, Recall, courbes ROC	✓ Complété
7	Interprétabilité : SHAP values, feature importance	✓ Complété
8	Recommandations métier et simulation individuelle	✓ Complété

Tableau 11 : Récapitulatif des étapes réalisées

V.2 Principaux enseignements

Output — Synthèse finale

```
================================================================
SYNTHESE FINALE - PROJET N14
Prediction du Turnover des Auditeurs Internes
ENCG Settat - Option CAC - Module IA
================================================================
MEILLEUR MODELE : Gradient Boosting
  AUC-ROC  : 0.7633
  Recall   : 0.4894 (capacite a detecter les departs reels)
  F1-Score : 0.4694

TOP 5 FACTEURS PREDICTIFS DU DEPART :
  1. StockOptionLevel
  2. JobSatisfaction
  3. EnvironmentSatisfaction
  4. JobLevel
  5. YearsWithCurrManager
```

- **Gradient Boosting comme meilleur modèle :** Avec un AUC-ROC de 0,7633, Gradient Boosting confirme la supériorité des méthodes de boosting pour les problèmes de classification déséquilibrée.
- **Facteurs prédictifs actionnables :** Les variables les plus déterminantes — StockOptionLevel, JobSatisfaction, EnvironmentSatisfaction, JobLevel, YearsWithCurrManager — sont directement sous le contrôle des directions RH, ce qui confère au modèle une valeur opérationnelle immédiate.
- **SMOTE comme étape essentielle :** La correction du déséquilibre 84 %/16 % par SMOTE s'est révélée indispensable pour permettre aux modèles d'apprendre correctement les patterns de départ.
- **SHAP pour la confiance décisionnelle :** L'interprétabilité via SHAP transforme un modèle opaque en un outil de décision transparent, essentiel pour obtenir l'adhésion des décideurs RH.

V.3 Perspectives d'amélioration

Ce travail ouvre plusieurs perspectives de recherche et d'application. À court terme, il serait pertinent d'optimiser les hyperparamètres par des techniques avancées (Optuna, Bayesian Optimization) et d'intégrer des données RH réelles issues de cabinets marocains. À moyen terme, le développement d'un **dashboard interactif** de monitoring du risque de turnover en temps réel constituerait une extension naturelle et à forte valeur ajoutée pour les décideurs RH.

> Ce projet illustre comment l'**Intelligence Artificielle** peut contribuer à transformer la gestion des ressources humaines d'une fonction réactive en une fonction **anticipatoire et stratégique**, en particulier dans des secteurs comme l'audit interne où le capital humain constitue l'actif le plus déterminant.

BIBLIOGRAPHIE / WEBOGRAPHIE

Dataset

IBM HR Analytics Employee Attrition & Performance Dataset — Kaggle.
Disponible sur : https://www.kaggle.com/datasets/pavansubhasht/ibm-hr-analytics-attrition-dataset

Références académiques

- [1] Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. *NeurIPS 2017.*
- [2] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *JAIR, 16, 321–357.*
- [3] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *KDD 2016.*
- [4] Ke, G., et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. *NeurIPS 2017.*
- [5] Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics, 29(5), 1189–1232.*

Bibliothèques Python utilisées

Bibliothèque	Version	Usage
pandas	≥ 1.5	Manipulation et exploration des données
numpy	≥ 1.23	Calcul numérique
scikit-learn	≥ 1.1	Prétraitement, modèles, évaluation
xgboost	≥ 1.7	Algorithme XGBoost
lightgbm	≥ 3.3	Algorithme LightGBM
imbalanced-learn	≥ 0.10	SMOTE — rééquilibrage des classes
shap	≥ 0.41	Interprétabilité des modèles
matplotlib	≥ 3.6	Visualisation graphique
seaborn	≥ 0.12	Visualisation statistique avancée

Tableau 12 : Environnement technique et bibliothèques utilisées

Environnement d'exécution : Google Colaboratory (Colab) — Python 3.10+ — Jupyter Notebook (.ipynb)

ANNEXES

Annexe A — Paramètres SMOTE et découpage Train/Test

Paramètre	Valeur
Ratio Train / Test	80 % / 20 % avec stratification
Méthode de rééquilibrage	SMOTE (imbalanced-learn)
Classe 0 après SMOTE (entraînement)	986 observations
Classe 1 après SMOTE (entraînement)	986 observations
Classe 0 sur test (réelle)	249 observations
Classe 1 sur test (réelle)	45 observations

Tableau 13 : Paramètres du découpage et du rééquilibrage SMOTE

Annexe B — Structure de la matrice de confusion

	Prédit Reste (0)	Prédit Départ (1)
Réel Reste (0)	Vrais Négatifs (VN)	Faux Positifs (FP)
Réel Départ (1)	Faux Négatifs (FN)	Vrais Positifs (VP)

Tableau 14 : Structure de la matrice de confusion — Gradient Boosting

Note : Les Faux Négatifs (départs non détectés) constituent le risque principal à minimiser dans ce contexte. Le modèle maintient un bon équilibre grâce à l'application du rééquilibrage SMOTE.

Annexe C — Métriques d'évaluation utilisées

Métrique	Formule	Pertinence dans ce contexte
Accuracy	(VP + VN) / Total	Vision globale — trompeuse sur données déséquilibrées
Precision	VP / (VP + FP)	Fiabilité des alertes de départ déclenchées
Recall	VP / (VP + FN)	**Capacité à détecter tous les départs — PRIORITAIRE**
F1-Score	Moyenne harmonique	Compromis équilibré Precision / Recall
AUC-ROC	Aire sous la courbe ROC	**Performance discriminante globale — MÉTRIQUE PRINCIPALE**

Tableau 15 : Métriques d'évaluation utilisées et leur pertinence